🤖 Spark MLlib — Machine Learning na dużą skalę

Gdy scikit-learn krzyczy "MemoryError!" 💥

Python w Zastosowaniach Big Data | Wykład 9

Prowadzący: Pavlo Melnyk

📋 Plan wykładu

1. Recap: co umiemy z scikit-learn (Wykład 8)
2. Dlaczego scikit-learn nie skaluje się na duże dane
3. Spark MLlib — architektura i filozofia
4. Przygotowanie danych: `VectorAssembler`
5. Kodowanie kategorii: `StringIndexer` i `OneHotEncoder`
6. `Pipeline` API — fit/transform na sterydach
7. Modele: `LogisticRegression` i `RandomForestClassifier`
8. Ewaluacja: `BinaryClassificationEvaluator`
9. scikit-learn vs MLlib — porównanie side-by-side
10. Najczęstsze pułapki
11. Podsumowanie i co dalej

> 🎯 **Po tym wykładzie:** Zbudujesz pełny pipeline ML w Spark MLlib — od surowych danych do wytrenowanego, wyewaluowanego modelu!

💡 Hook — Dlaczego tu jesteśmy?

Na Wykładzie 8 nauczyliście się scikit-learn: `fit()`, `predict()`, `Pipeline`, `ColumnTransformer`. Wszystko działa świetnie... dopóki dane mieszczą się w RAM jednego laptopa 💻

```
# scikit-learn na 50 000 wierszy — pyk i gotowe! 🎉
from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train, y_train)
```

```
# scikit-learn na 300 000 000 wierszy logów reklamowych... 💀
model.fit(X_train_ads, y_train_ads)
# MemoryError: Unable to allocate 128. GiB for an array
```

> Reklamodawcy generują **miliardy** wyświetleń dziennie. Czas poznać **Spark MLlib**! 🔥

🔄 Recap: Co umiemy z scikit-learn?

Na Wykładzie 8 poznaliśmy pełny warsztat ML na jednej maszynie:

Koncept	scikit-learn	Status
Preprocessing	`SimpleImputer`, `StandardScaler`	✅ Umiemy!
Encoding	`OneHotEncoder`, `LabelEncoder`	✅ Umiemy!
Pipeline	`sklearn.pipeline.Pipeline`	✅ Umiemy!
Modele	`LogisticRegression`, `RandomForestClassifier`	✅ Umiemy!
Ewaluacja	`accuracy_score`, `roc_auc_score`	✅ Umiemy!
Train/Test Split	`train_test_split()`	✅ Umiemy!

> 💪 Dziś **przenosimy** ten sam sposób myślenia na klaster Spark!

⚠️ Problem: pamięć jednej maszyny to limit

scikit-learn wymaga, by **cały** zbiór danych (i macierz features) zmieścił się w RAM procesu Pythona:

Scenariusz	Rozmiar danych	scikit-learn	MLlib
Ankieta studencka	5 MB	✅ Instant	⚡ Overkill
Sklep internetowy	2 GB	✅ 1 min	✅ 2 min
Logi wyświetleń reklam (1 dzień)	50 GB	💀 OOM	✅ 5 min
Logi wyświetleń reklam (1 miesiąc)	1.5 TB	🪦 Nieosiągalne	✅ 40 min
Historia kliknięć (1 rok, globalnie)	20 TB	🪦 Nieosiągalne	✅ kilka godzin

> 🔑 **Zasada kciuka:** Dane < 10 GB → scikit-learn. Dane > 10 GB (albo rosną) → **MLlib**!

🤔 Think About It

Pytanie: Sieć reklamowa wyświetla **2 miliardy** reklam dziennie. Każdy rekord to ok. 200 bajtów (kategoria, urządzenie, pozycja, timestamp...).

- Ile to danych **dziennie**? A w skali **miesiąca**?
- Czy Twój laptop z 16 GB RAM da radę wczytać to do pandas?

> To właśnie scenariusz, w którym **Spark MLlib** ratuje sytuację! 🦸

🚀 Czym jest Spark MLlib?

MLlib (a dokładniej `pyspark.ml`) to biblioteka Machine Learning wbudowana w Spark, zaprojektowana od zera do pracy **rozproszonej**.

Cecha	scikit-learn	Spark MLlib
Skala	Jedna maszyna (RAM)	Klaster (TB+)
Struktura danych	pandas DataFrame / NumPy array	Spark DataFrame
API	`fit()` / `predict()`	`fit()` / `transform()`
Feature prep	`ColumnTransformer`	`VectorAssembler`
Wykonanie	Jeden proces	Rozproszone (lazy evaluation)
Modele	Dziesiątki algorytmów	Mniejszy, ale rosnący zestaw

> 💡 **Analogia:** scikit-learn to szefowa kuchni gotująca sama. MLlib to cała armia kucharzy w wielkiej restauracji — każdy przygotowuje swoją część dania równolegle! 👨‍🍳👨‍🍳

🧱 Architektura MLlib (spark.ml)

MLlib operuje wyłącznie na **Spark DataFrame** — nie na RDD (to stary, przestarzały `mllib` package, którego już nie używamy).

Kluczowe komponenty:

- **Transformer** — przekształca DataFrame w inny DataFrame (np. `VectorAssembler.transform()`)
- **Estimator** — algorytm, który uczy się na danych i produkuje Transformer (np. `LogisticRegression.fit()` → `LogisticRegressionModel`)
- **Pipeline** — łańcuch Transformerów i Estimatorów wykonywanych sekwencyjnie
- **Evaluator** — mierzy jakość modelu na podstawie kolumny z predykcjami

> 🔑 Zapamiętaj: **Estimator.fit() → Transformer**, **Transformer.transform() → DataFrame**

🔧 Przygotowanie danych: `VectorAssembler`

MLlib wymaga, aby wszystkie features były połączone w **jedną kolumnę** typu wektor — to zastępuje macierz `X` z scikit-learn.

```
from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(
    inputCols=["wiek_grupy", "pozycja_num", "godzina"],
    outputCol="features_raw"
)

df_with_features = assembler.transform(df)
df_with_features.select("features_raw").show(3, truncate=False)
```

```
+--------------------+
|features_raw        |
+--------------------+
|[2.0, 1.0, 14.0]    |
|[1.0, 0.0, 9.0]     |
|[3.0, 2.0, 21.0]    |
+--------------------+
```

> ⚠️ `VectorAssembler` **nie** koduje zmiennych kategorycznych typu string — potrzebujemy do tego `StringIndexer`!

🔤 Kodowanie kategorii: `StringIndexer`

`StringIndexer` zamienia string na indeks liczbowy (podobnie do `LabelEncoder` w scikit-learn):

```
from pyspark.ml.feature import StringIndexer

indexer = StringIndexer(
    inputCol="typ_urzadzenia",
    outputCol="typ_urzadzenia_idx"
)

model = indexer.fit(df)
indexed = model.transform(df)
indexed.select("typ_urzadzenia", "typ_urzadzenia_idx").distinct().show()
```

```
+---------------+------------------+
|typ_urzadzenia |typ_urzadzenia_idx|
+---------------+------------------+
|mobile         |0.0               |
|desktop        |1.0               |
|tablet         |2.0               |
+---------------+------------------+
```

> ⚠️ Indeksy 0, 1, 2 sugerują **kolejność** — modele liniowe mogą to źle zinterpretować. Wchodzi `OneHotEncoder`!

🎯 One-Hot Encoding w MLlib

```
from pyspark.ml.feature import OneHotEncoder

encoder = OneHotEncoder(
    inputCol="typ_urzadzenia_idx",
    outputCol="typ_urzadzenia_ohe"
)

encoded = encoder.fit(indexed).transform(indexed)
encoded.select("typ_urzadzenia_idx", "typ_urzadzenia_ohe").show(3)
```

```
+------------------+------------------+
|typ_urzadzenia_idx|typ_urzadzenia_ohe|
+------------------+------------------+
|0.0               |(2,[0],[1.0])     |
|1.0               |(2,[1],[1.0])     |
|2.0               |(2,[],[])         |
+------------------+------------------+
```

> 💡 To `SparseVector` — kategoria "tablet" (ostatnia) jest zakodowana jako wektor zerowy, by uniknąć kolinearności (jak `drop='first'` w scikit-learn).

🔗 Pipeline API — łączymy wszystko

Zamiast ręcznie wywoływać każdy krok, budujemy **Pipeline** — dokładnie jak `sklearn.pipeline.Pipeline`, ale na sterydach rozproszonych:

```
from pyspark.ml import Pipeline
from pyspark.ml.classification import LogisticRegression

pipeline = Pipeline(stages=[
    StringIndexer(inputCol="typ_urzadzenia", outputCol="typ_urzadzenia_idx"),
    OneHotEncoder(inputCol="typ_urzadzenia_idx", outputCol="typ_urzadzenia_ohe"),
    VectorAssembler(
        inputCols=["typ_urzadzenia_ohe", "pozycja_num", "wiek_grupy_idx"],
        outputCol="features"
    ),
    LogisticRegression(featuresCol="features", labelCol="clicked")
])

model = pipeline.fit(train_df)          # 👈 trenuje CAŁY łańcuch
predictions = model.transform(test_df)  # 👈 transformuje testowe dane
```

> 🎯 `pipeline.fit()` zwraca `PipelineModel` — gotowy do `transform()` na nowych danych, np. w produkcji!

🌲 Modele: `LogisticRegression` i `RandomForestClassifier`

```python
from pyspark.ml.classification import LogisticRegression, RandomForestClassifier

# Model liniowy – szybki, interpretowalny
lr = LogisticRegression(featuresCol="features", labelCol="clicked", maxIter=20)

# Ensemble drzew – zwykle wyższa jakość, wolniejszy trening
rf = RandomForestClassifier(
    featuresCol="features", labelCol="clicked", numTrees=100, maxDepth=8
)
```

Model	Zalety	Wady
`LogisticRegression`	Szybki, prosty, interpretowalny	Zakłada liniowość granic decyzyjnych
`RandomForestClassifier`	Wychwytuje nieliniowości, odporny na outliery	Wolniejszy, więcej pamięci

> 💡 W praktyce: zacznij od `LogisticRegression` jako baseline, potem porównaj z `RandomForestClassifier`.

📊 Ewaluacja: `BinaryClassificationEvaluator`

```python
from pyspark.ml.evaluation import BinaryClassificationEvaluator

evaluator = BinaryClassificationEvaluator(
    labelCol="clicked",
    rawPredictionCol="rawPrediction",
    metricName="areaUnderROC"
)

auc = evaluator.evaluate(predictions)
print(f"AUC: {auc:.4f}")
# AUC: 0.8123
```

> 🎯 **AUC (Area Under ROC Curve)** — im bliżej 1.0, tym lepiej model odróżnia kliknięcia od braku kliknięcia. 0.5 = losowe zgadywanie!

Dla dokładniejszej analizy (accuracy, precision, recall) używamy `MulticlassClassificationEvaluator` — działa też dla klasyfikacji binarnej.

⚖️ scikit-learn vs MLlib — side by side

```
# scikit-learn
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.linear_model import LogisticRegression

preprocess = ColumnTransformer([
    ("ohe", OneHotEncoder(), ["typ_urzadzenia"])
])
pipe = Pipeline([("prep", preprocess), ("model", LogisticRegression())])
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
```

```
# Spark MLlib
from pyspark.ml import Pipeline
from pyspark.ml.feature import StringIndexer, OneHotEncoder, VectorAssembler
from pyspark.ml.classification import LogisticRegression

pipeline = Pipeline(stages=[StringIndexer(...), OneHotEncoder(...),
                            VectorAssembler(...), LogisticRegression(...)])
model = pipeline.fit(train_df)
predictions = model.transform(test_df)
```

🔑 Ten sam **mental model** — Pipeline, fit, transform/predict — inna infrastruktura pod spodem!

⚠️ Najczęstsze pułapki

Pułapka	Dlaczego to problem	Rozwiązanie
Zapomniany `VectorAssembler`	Model nie przyjmuje "gołych" kolumn	Zawsze łącz features w jedną kolumnę wektorową
`StringIndexer` bez `OneHotEncoder`	Model liniowy widzi fałszywą kolejność kategorii	Dodaj `OneHotEncoder` po `StringIndexer`
`fit()` na całym zbiorze przed splitem	Data leakage — model "widział" dane testowe	Zawsze `randomSplit()` **przed** `fit()` na Pipeline
Ignorowanie `rawPrediction` vs `prediction`	Evaluator AUC wymaga `rawPrediction`, nie etykiety 0/1	Sprawdź nazwę kolumny wymaganą przez evaluator
Zbyt mały `numTrees` / `maxDepth` na dużych danych	Underfitting mimo dużego zbioru	Testuj kilka konfiguracji, monitoruj AUC na zbiorze testowym

Key Takeaways

1. ✅ scikit-learn ma limit — RAM jednej maszyny; MLlib skaluje się na klaster
2. ✅ MLlib operuje na Spark DataFrame przez `Transformer` i `Estimator`
3. ✅ `VectorAssembler` łączy features w jedną kolumnę wektorową
4. ✅ `Pipeline` łączy preprocessing i model w jeden łańcuch fit/transform
5. ✅ `BinaryClassificationEvaluator` (AUC) to standardowa metryka klasyfikacji binarnej

🔮 Co dalej?

Na dzisiejszych **ćwiczeniach** zbudujesz pełny pipeline MLlib do predykcji **kliknięć w reklamy (CTR)** na ~300 000 wyświetleń! 📱🖱️

Wykład 10 (08.12.2026): Zaawansowany MLlib — tuning hiperparametrów (`ParamGridBuilder` , `CrossValidator`)!

❓ Pytania?

Dziękuję za uwagę! 🎉

Zajęcia nr 9 | 01.12.2026